

15.072 Deliverable 1

Alessandro Ferraiolo, Carlo Calvanico, Chris Low, Reina Wu

Problem_1

September 23, 2026

1 Problem 1

1.1 (a)

```
[10]: import pandas as pd
import numpy as np
import statsmodels.formula.api as smf

from statsmodels.stats.anova import anova_lm
from statsmodels.stats.outliers_influence import variance_inflation_factor
from patsy import dmatrices

train = pd.read_csv("laptop_train.csv")
test = pd.read_csv("laptop_test.csv")

train.head()
```

```
[10]:
```

	InventoryID	Company	TypeName	GPU	Screen	Memory	Weight	Rating	\
0	6	Asus	Gaming	Nvidia	17.3	16	2.90	1	
1	15	Asus	Gaming	Nvidia	17.3	16	2.73	3	
2	17	Asus	Gaming	Nvidia	15.6	16	2.50	4	
3	18	Asus	Gaming	Nvidia	17.3	16	4.00	10	
4	38	Asus	Gaming	Nvidia	15.6	8	2.30	6	

	Price
0	2122.0
1	2049.9
2	1799.0
3	998.0
4	1649.0

```
[11]: train.info()

train[["Screen", "Memory", "Weight", "Rating", "Price"]].describe()

for col in ["Company", "TypeName", "GPU"]:
    print(col, train[col].unique())
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 665 entries, 0 to 664
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   InventoryID     665 non-null   int64
1   Company         665 non-null   object
2   TypeName        665 non-null   object
3   GPU             665 non-null   object
4   Screen          665 non-null   float64
5   Memory          665 non-null   int64
6   Weight          665 non-null   float64
7   Rating          665 non-null   int64
8   Price           665 non-null   float64
dtypes: float64(3), int64(3), object(3)
memory usage: 46.9+ KB
Company ['Asus' 'Dell' 'HP' 'Lenovo']
TypeName ['Gaming' 'Notebook' 'Ultrabook']
GPU ['Nvidia' 'Intel' 'AMD']

```

```
[12]: train[["Screen", "Memory", "Weight", "Rating", "Price"]].corr()
```

```

[12]:      Screen    Memory    Weight    Rating    Price
Screen  1.000000  0.099408  0.811503 -0.030974 -0.130666
Memory  0.099408  1.000000  0.292397  0.024310  0.722416
Weight  0.811503  0.292397  1.000000 -0.019247  0.116779
Rating -0.030974  0.024310 -0.019247  1.000000 -0.029004
Price  -0.130666  0.722416  0.116779 -0.029004  1.000000

```

```

[13]: full_formula = (
      "Price ~ C(Company) + C(TypeName) + C(GPU) "
      "+ Screen + Memory + Weight + Rating"
    )

full_model = smf.ols(full_formula, data=train).fit()

print(full_model.summary())

```

OLS Regression Results

```

=====
Dep. Variable:          Price    R-squared:                0.660
Model:                  OLS      Adj. R-squared:            0.654
Method:                 Least Squares    F-statistic:           115.1
Date:                  Mon, 21 Sep 2026    Prob (F-statistic):    8.75e-145
Time:                  11:20:15    Log-Likelihood:        -4809.1
No. Observations:      665    AIC:                   9642.
Df Residuals:          653    BIC:                   9696.
Df Model:               11

```

```

Covariance Type:      nonrobust
=====
=====
              coef      std err          t      P>|t|      [0.025
0.975]
-----
Intercept          1434.2466    288.621     4.969     0.000     867.510
2000.984
C(Company) [T.Dell]      86.7254     42.931     2.020     0.044      2.426
171.025
C(Company) [T.HP]       170.3954     43.502     3.917     0.000     84.974
255.816
C(Company) [T.Lenovo]    35.3279     42.129     0.839     0.402    -47.396
118.052
C(TypeName) [T.Notebook] -76.2954     58.721    -1.299     0.194   -191.600
39.009
C(TypeName) [T.Ultrabook] 416.3939     74.848     5.563     0.000    269.421
563.366
C(GPU) [T.Intel]        165.4297     39.663     4.171     0.000     87.547
243.312
C(GPU) [T.Nvidia]       218.1386     46.457     4.696     0.000    126.916
309.361
Screen              -120.9323     21.801    -5.547     0.000   -163.741
-78.123
Memory               87.3725      4.398    19.866     0.000     78.736
96.009
Weight              271.0986     49.441     5.483     0.000    174.017
368.180
Rating              -11.8206      4.613    -2.563     0.011    -20.878
-2.763
=====
Omnibus:              78.159   Durbin-Watson:              1.798
Prob(Omnibus):         0.000   Jarque-Bera (JB):          152.765
Skew:                  0.704   Prob(JB):                  6.72e-34
Kurtosis:              4.880   Cond. No.                  406.
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```

[14]: # Check whether all predictors contribute meaningfully to the model.
terms = [
    "C(Company)",
    "C(TypeName)",
    "C(GPU)",

```

```

    "Screen",
    "Memory",
    "Weight",
    "Rating"
]

joint_pvalues = {}

for term in terms:
    reduced_terms = [x for x in terms if x != term]
    reduced_formula = "Price ~ " + " + ".join(reduced_terms)
    reduced_model = smf.ols(reduced_formula, data=train).fit()

    comparison = anova_lm(reduced_model, full_model)
    joint_pvalues[term] = comparison["Pr(>F)"].iloc[1]

pd.Series(joint_pvalues, name="p-value").sort_values()

```

```

[14]: Memory          4.733674e-69
      C(TypeName)     7.913499e-23
      Screen         4.223609e-08
      Weight         5.971125e-08
      C(GPU)         3.623031e-06
      C(Company)     1.765951e-04
      Rating        1.060948e-02
      Name: p-value, dtype: float64

```

```

[15]: # Check for highly correlated predictors (Screen and Weight) that may create
      ↪ multicollinearity.
      _, X = dmatrices(full_formula, data=train, return_type="dataframe")

      vif = pd.DataFrame({
          "Variable": X.columns,
          "VIF": [
              variance_inflation_factor(X.values, i)
              for i in range(X.shape[1])
          ]
      })

      vif

```

```

[15]:

```

	Variable	VIF
0	Intercept	486.079037
1	C(Company) [T.Dell]	2.208569
2	C(Company) [T.HP]	2.156861
3	C(Company) [T.Lenovo]	2.086471
4	C(TypeName) [T.Notebook]	4.278847

5	C(TypeName) [T.Ultrabook]	4.346600
6	C(GPU) [T.Intel]	2.283444
7	C(GPU) [T.Nvidia]	2.602129
8	Screen	3.857381
9	Memory	1.603762
10	Weight	5.246611
11	Rating	1.009567

```
[16]: # Compare the full model with two reduced alternatives
model_no_weight = smf.ols(
    "Price ~ C(Company) + C(TypeName) + C(GPU) "
    "+ Screen + Memory + Rating",
    data=train
).fit()

model_no_screen = smf.ols(
    "Price ~ C(Company) + C(TypeName) + C(GPU) "
    "+ Memory + Weight + Rating",
    data=train
).fit()

def oos_r2(model):
    y_test = test["Price"]
    y_pred = model.predict(test)

    ssr_model = np.sum((y_test - y_pred) ** 2)
    ssr_baseline = np.sum((y_test - train["Price"].mean()) ** 2)

    return 1 - ssr_model / ssr_baseline

def test_rmse(model):
    y_test = test["Price"]
    y_pred = model.predict(test)

    return np.sqrt(np.mean((y_test - y_pred) ** 2))

comparison = pd.DataFrame({
    "Model": ["Full model", "Drop Weight", "Drop Screen"],
    "Training R2": [
        full_model.rsquared,
        model_no_weight.rsquared,
        model_no_screen.rsquared
    ],
    "OOS R2": [
```

```

        oos_r2(full_model),
        oos_r2(model_no_weight),
        oos_r2(model_no_screen)
    ],
    "Test RMSE": [
        test_rmse(full_model),
        test_rmse(model_no_weight),
        test_rmse(model_no_screen)
    ]
})

comparison

```

```

[16]:
      Model  Training R2    OOS R2   Test RMSE
0  Full model    0.659727  0.551638  357.005261
1  Drop Weight    0.644059  0.523020  368.222225
2  Drop Screen    0.643693  0.525559  367.240962

```

1.1.1 Model selection procedure

I started with the laptop characteristics that seemed relevant for explaining price (excluding `InventoryID` because it is an internal stocking identifier rather than a product characteristic). `Company`, `TypeName`, and `GPU` were treated as categorical variables. I assessed the significance of categorical predictors jointly using partial F-tests, and all terms in the model were significant at the 5% level.

I also checked for multicollinearity among the predictors. `Screen` and `Weight` had a relatively high correlation (about 0.81), although the largest non-intercept VIF was about 5.25. I therefore considered reduced models dropping each variable separately. Both reduced models performed worse on the test set, so I retained both variables. The final specification was selected based on statistical significance, multicollinearity considerations, and out-of-sample predictive performance.

```

[17]: final_model = full_model

print(final_model.summary())

```

```

                                OLS Regression Results
=====
Dep. Variable:                  Price    R-squared:                0.660
Model:                            OLS    Adj. R-squared:            0.654
Method:                 Least Squares    F-statistic:                115.1
Date:                Mon, 21 Sep 2026    Prob (F-statistic):        8.75e-145
Time:                  11:20:16    Log-Likelihood:            -4809.1
No. Observations:                665    AIC:                        9642.
Df Residuals:                    653    BIC:                        9696.
Df Model:                          11
Covariance Type:                  nonrobust
=====
=====

```

	coef	std err	t	P> t	[0.025
0.975]					

Intercept	1434.2466	288.621	4.969	0.000	867.510
2000.984					
C(Company) [T.Dell]	86.7254	42.931	2.020	0.044	2.426
171.025					
C(Company) [T.HP]	170.3954	43.502	3.917	0.000	84.974
255.816					
C(Company) [T.Lenovo]	35.3279	42.129	0.839	0.402	-47.396
118.052					
C(TypeName) [T.Notebook]	-76.2954	58.721	-1.299	0.194	-191.600
39.009					
C(TypeName) [T.Ultrabook]	416.3939	74.848	5.563	0.000	269.421
563.366					
C(GPU) [T.Intel]	165.4297	39.663	4.171	0.000	87.547
243.312					
C(GPU) [T.Nvidia]	218.1386	46.457	4.696	0.000	126.916
309.361					
Screen	-120.9323	21.801	-5.547	0.000	-163.741
-78.123					
Memory	87.3725	4.398	19.866	0.000	78.736
96.009					
Weight	271.0986	49.441	5.483	0.000	174.017
368.180					
Rating	-11.8206	4.613	-2.563	0.011	-20.878
-2.763					
=====					
Omnibus:	78.159	Durbin-Watson:			1.798
Prob(Omnibus):	0.000	Jarque-Bera (JB):			152.765
Skew:	0.704	Prob(JB):			6.72e-34
Kurtosis:	4.880	Cond. No.			406.
=====					

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

1.2 (b)

Several variables in the final model have effects that are managerially sensible to me. **Memory** has a positive and statistically significant coefficient, which is consistent with higher-memory laptops being more expensive. **TypeName** and **GPU** also show meaningful price differences across categories: Ultrabooks are priced higher than Gaming laptops, while Intel and Nvidia GPUs have positive effects relative to AMD GPUs, holding the other variables fixed. **Company** is also sensible to include, since manufacturers may have different price levels even after controlling for the observed laptop characteristics.

I would investigate **Screen**, **Weight**, and **Rating** further. **Screen** has a negative coefficient while **Weight** has a positive coefficient, which is not immediately intuitive. Since **Screen** and **Weight** are also fairly highly correlated in the training data, their individual coefficients should be interpreted with some caution. **Rating** is statistically significant but has a small negative coefficient, which is also somewhat surprising and would be worth examining further.

1.3 (c)

HP has the largest estimated manufacturer effect on price, while Asus has the smallest. Holding the other laptop characteristics fixed, HP laptops are predicted to cost about €170 more than otherwise comparable Asus laptops. With Asus as the reference category, the estimated manufacturer effects are €0 for Asus, approximately €87 for Dell, €170 for HP, and €35 for Lenovo.

1.4 (d)

```
[27]: final_oos_r2 = oos_r2(final_model)
      print("Out-of-sample R-squared:", final_oos_r2)
```

Out-of-sample R-squared: 0.5516377499322669

The out-of-sample R^2 of the final model is approximately 0.552. This means that, on the test set, the model reduces squared prediction error by about 55.2% relative to the baseline model that predicts the average laptop price in the training set for every test observation.

1.5 (e)

```
[20]: from scipy.stats import norm

new_laptop = pd.DataFrame({
    "Company": ["Asus"],
    "TypeName": ["Ultrabook"],
    "GPU": ["Intel"],
    "Screen": [15.6],
    "Memory": [6],
    "Weight": [3],
    "Rating": [8]
})

predicted_price = final_model.predict(new_laptop).iloc[0]
rse = final_model.scale ** 0.5

prob_above_1100 = 1 - norm.cdf(
    1100,
    loc=predicted_price,
    scale=rse
)

print("Predicted price:", predicted_price)
print("Residual standard error:", rse)
```

```
print("Probability price exceeds 1100:", prob_above_1100)
```

Predicted price: 1372.4919244244215
 Residual standard error: 337.58653694113485
 Probability price exceeds 1100: 0.7902176029277714

For this laptop, the final model gives a point prediction of around €1,372.49. I assume that **the regression errors are independent and approximately normally distributed with mean zero and constant variance**, and that **uncertainty from estimating the regression coefficients is small relative to the residual variation**. Using the model's residual standard error of approximately €337.59, I treat the price prediction as approximately

$$Price \sim N(1372.49, 337.59^2).$$

Therefore,

$$P(Price > 1100) = 1 - \Phi\left(\frac{1100 - 1372.49}{337.59}\right) \approx 0.790.$$

Thus, under these assumptions, the probability that the laptop's price exceeds €1,100 is approximately 79.0%.

1.6 (f)

```
[25]: interaction_model_f = smf.ols(
      "Price ~ C(Company) + C(TypeName) + Screen + Weight + Rating + Memory *␣
      ↪C(GPU)",
      data=train
    ).fit()

print(interaction_model_f.summary())
print("Part (a) OOS R2:", oos_r2(final_model))
print("Part (f) OOS R2:", oos_r2(interaction_model_f))
```

OLS Regression Results

```
=====
Dep. Variable:          Price    R-squared:                0.665
Model:                  OLS      Adj. R-squared:            0.659
Method:                 Least Squares    F-statistic:          99.60
Date:                  Tue, 22 Sep 2026    Prob (F-statistic):    4.32e-145
Time:                  23:42:24    Log-Likelihood:        -4803.4
No. Observations:      665      AIC:                  9635.
Df Residuals:          651      BIC:                  9698.
Df Model:               13
Covariance Type:       nonrobust
=====
```

	coef	std err	t	P> t	[0.025
--	------	---------	---	------	--------

0.975]

Intercept	1635.8257	293.069	5.582	0.000	1060.351
2211.301					
C(Company) [T.Dell]	82.8037	42.702	1.939	0.053	-1.046
166.653					
C(Company) [T.HP]	160.9213	43.296	3.717	0.000	75.905
245.938					
C(Company) [T.Lenovo]	33.3374	41.937	0.795	0.427	-49.010
115.685					
C(TypeName) [T.Notebook]	-72.5664	59.035	-1.229	0.219	-188.488
43.356					
C(TypeName) [T.Ultrabook]	395.3125	76.346	5.178	0.000	245.399
545.226					
C(GPU) [T.Intel]	-92.4312	88.996	-1.039	0.299	-267.185
82.323					
C(GPU) [T.Nvidia]	-29.5910	99.965	-0.296	0.767	-225.883
166.701					
Screen	-120.3982	21.728	-5.541	0.000	-163.064
-77.733					
Weight	271.2845	50.406	5.382	0.000	172.307
370.262					
Rating	-11.9920	4.582	-2.617	0.009	-20.990
-2.995					
Memory	58.3904	9.763	5.981	0.000	39.220
77.561					
Memory:C(GPU) [T.Intel]	37.2253	11.721	3.176	0.002	14.209
60.242					
Memory:C(GPU) [T.Nvidia]	32.9786	11.464	2.877	0.004	10.467
55.490					
=====					
Omnibus:	80.193	Durbin-Watson:		1.790	
Prob(Omnibus):	0.000	Jarque-Bera (JB):		163.828	
Skew:	0.704	Prob(JB):		2.66e-36	
Kurtosis:	4.982	Cond. No.		433.	
=====					

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Part (a) OOS R2: 0.5516377499322669

Part (f) OOS R2: 0.5507221553514837

Since AMD is the reference GPU category here, the coefficient on Memory (58.39) represents the estimated effect of an additional GB of memory for laptops with AMD GPUs. The interaction coefficients show how this effect changes for the other GPU types: for Intel GPUs, the effect of an additional GB of memory is about €37.23 larger than for AMD GPUs, giving a total effect of

about €95.62 per GB; for Nvidia GPUs, it is approximately €32.98 larger, giving a total effect of about €91.37 per GB.

Compared with the model in part (a), this specification allows the effect of memory on price to vary across GPU types rather than assuming a common memory effect for all laptops. Its in-sample R^2 increases slightly from 0.660 to 0.665, while its out-of-sample R^2 is essentially unchanged and slightly lower, decreasing from 0.5516 to 0.5507.

1.7 (g)

```
[26]: interaction_model_g = smf.ols(
    "Price ~ C(TypeName) + Screen + Memory + Weight + Rating + C(GPU) *  

    ↪C(Company)",
    data=train
).fit()

print(interaction_model_g.summary())
print("Part (a) OOS R2:", oos_r2(final_model))
print("Part (g) OOS R2:", oos_r2(interaction_model_g))
```

OLS Regression Results				
=====				
Dep. Variable:	Price	R-squared:	0.676	
Model:	OLS	Adj. R-squared:	0.668	
Method:	Least Squares	F-statistic:	79.56	
Date:	Tue, 22 Sep 2026	Prob (F-statistic):	8.41e-146	
Time:	23:42:49	Log-Likelihood:	-4792.3	
No. Observations:	665	AIC:	9621.	
Df Residuals:	647	BIC:	9702.	
Df Model:	17			
Covariance Type:	nonrobust			
=====				
=====				
		coef	std err	t
P> t	[0.025	0.975]		

Intercept		1710.5748	309.677	5.524
0.000	1102.482	2318.667		
C(TypeName) [T.Notebook]		-15.8465	59.018	-0.269
0.788	-131.736	100.043		
C(TypeName) [T.Ultrabook]		488.9254	74.667	6.548
0.000	342.307	635.543		
C(GPU) [T.Intel]		-279.7797	138.648	-2.018
0.044	-552.035	-7.524		
C(GPU) [T.Nvidia]		-45.8472	132.522	-0.346
0.729	-306.073	214.379		
C(Company) [T.Dell]		-286.9438	135.710	-2.114

0.035	-553.428	-20.459			
C(Company) [T.HP]			-170.4844	140.795	-1.211
0.226	-446.955	105.986			
C(Company) [T.Lenovo]			-322.1718	148.715	-2.166
0.031	-614.194	-30.150			
C(GPU) [T.Intel]:C(Company) [T.Dell]			423.9139	148.829	2.848
0.005	131.667	716.161			
C(GPU) [T.Nvidia]:C(Company) [T.Dell]			477.2737	150.926	3.162
0.002	180.909	773.638			
C(GPU) [T.Intel]:C(Company) [T.HP]			464.5333	152.415	3.048
0.002	165.245	763.822			
C(GPU) [T.Nvidia]:C(Company) [T.HP]			150.2330	159.453	0.942
0.346	-162.875	463.341			
C(GPU) [T.Intel]:C(Company) [T.Lenovo]			508.6057	160.102	3.177
0.002	194.223	822.988			
C(GPU) [T.Nvidia]:C(Company) [T.Lenovo]			229.8932	159.225	1.444
0.149	-82.767	542.553			
Screen			-118.4726	21.867	-5.418
0.000	-161.412	-75.533			
Memory			86.1194	4.327	19.901
0.000	77.622	94.617			
Weight			259.0412	49.048	5.281
0.000	162.729	355.353			
Rating			-11.5912	4.532	-2.558
0.011	-20.491	-2.692			
=====					
Omnibus:	65.207	Durbin-Watson:		1.776	
Prob(Omnibus):	0.000	Jarque-Bera (JB):		135.523	
Skew:	0.581	Prob(JB):		3.73e-30	
Kurtosis:	4.882	Cond. No.		665.	
=====					

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Part (a) OOS R²: 0.5516377499322669

Part (g) OOS R²: 0.5640604425363809

Since Asus and AMD are the reference categories, each interaction coefficient measures the additional price effect for a particular Company-GPU combination beyond the corresponding main effects. For example, the coefficient on C(GPU) [T.Intel]:C(Company) [T.Dell] is approximately 423.91, meaning that, holding the other characteristics fixed, the effect of an Intel GPU on price is about €423.91 larger for Dell laptops than for Asus laptops.

Compared with the model in part (a), this specification allows the effect of GPU on price to vary across manufacturers rather than assuming the same GPU effect for every company. The interaction model has a slightly higher in-sample R^2 (0.676 versus 0.660) and a slightly higher out-of-sample R^2 (0.564 versus 0.552).

Problem 2 - Health insurance charges: distribution fitting and inference.

(a) Load the data

```
In [28]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
import statsmodels.formula.api as smf
from scipy import stats

ins = pd.read_csv("insurance.csv")
ins.head()
```

```
Out[28]:
```

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520

1,338 policyholders × 7 columns (age , sex , bmi , children , smoker , region , charges), no missing values.

(b) Mean and sample standard deviation

```
In [29]: mu, sd = ins["charges"].mean(), ins["charges"].std()
print(f"mean          = ${mu:,.2f}")
print(f"sample st. dev. = ${sd:,.2f}")
```

```
mean          = $13,270.42
sample st. dev. = $12,110.01
```

The standard deviation is nearly as large as the mean, this is a first sign the variable is not normal.

(c) Probability of a charge between \$8,000 and \$14,000, assuming normality

```
In [30]: p_norm = stats.norm.cdf(14000, loc=mu, scale=sd) - stats.norm.cdf(8000, loc=mu,
scale=sd)
```

```
print(f"Phi(14000) = {stats.norm.cdf(14000, mu, sd):.4f}")
print(f"Phi(8000) = {stats.norm.cdf(8000, mu, sd):.4f}")
print(f"P(8000 < X < 14000) = {p_norm:.4f}")
```

```
Phi(14000) = 0.5240
Phi(8000) = 0.3317
P(8000 < X < 14000) = 0.1923
```

$$P(8000 < X < 14000) = \Phi\left(\frac{14000-13270.42}{12110.01}\right) - \Phi\left(\frac{8000-13270.42}{12110.01}\right) = 0.5240 - 0.3317 = \mathbf{0.1923}$$

Under a Normal(13270.42, 12110.01) model, **19.2%** of policyholders would fall in this window.

(d) The same probability from the data

```
In [31]: inwin = ins["charges"].between(8000, 14000)
print(f"empirical proportion = {inwin.mean():.4f} ({inwin.sum()} of {len(ins)})")
print(f"normal model = {p_norm:.4f}")
print(f"\nskewness = {stats.skew(ins['charges']):.3f}")
print(f"P(charges < 0) implied by the fitted normal = {stats.norm.cdf(0, mu, sd):.4f}")
print("\nmean charge by smoker status:")
print(ins.groupby("smoker")["charges"].agg(["mean", "count"]).round(1))
```

```
empirical proportion = 0.2825 (378 of 1338)
normal model = 0.1923
```

```
skewness = 1.514
P(charges < 0) implied by the fitted normal = 0.1366
```

```
mean charge by smoker status:
      mean  count
smoker
no      8434.3  1064
yes     32050.2   274
```

378 of 1,338 policyholders (28.25%) actually fall in the window, against 19.2% under the normal model. **It does not align:** the normal understates by ~9 percentage points, about a third too little mass. Charges are not normal:

- **Strongly right-skewed** (skewness 1.51).
- **Bimodal:** the data are a mixture of two populations — non-smokers average \$8,434 ($n = 1,064$), smokers \$32,050 ($n = 274$). One symmetric bell cannot represent two clusters.
- **σ is inflated to cover the tail.** Reaching charges of \$63,770 requires $\sigma = \$12,110$, which flattens the density in the middle and drains mass from the \$8k–\$14k window.
- **It assigns impossible values:** the fitted normal puts **13.7%** of its probability on *negative* charges, meaning that part of this model is totally wrong, not just imprecise.

(e) Comparing four candidate distributions

```
In [32]: charges = ins["charges"].values

fits = {
```

```

    "normal": (stats.norm, stats.norm.fit(charges)),
    "lognormal": (stats.lognorm, stats.lognorm.fit(charges)),
    "uniform": (stats.uniform, stats.uniform.fit(charges)),
    "Cauchy": (stats.cauchy, stats.cauchy.fit(charges)),
}

rows = []
for name, (dist, par) in fits.items():
    rows.append({
        "distribution": name,
        "parameters": ", ".join(f"{v:.4g}" for v in par),
        "log-likelihood": np.sum(dist.logpdf(charges, *par)),
    })
pd.DataFrame(rows).set_index("distribution").round(4)

```

Out[32]:

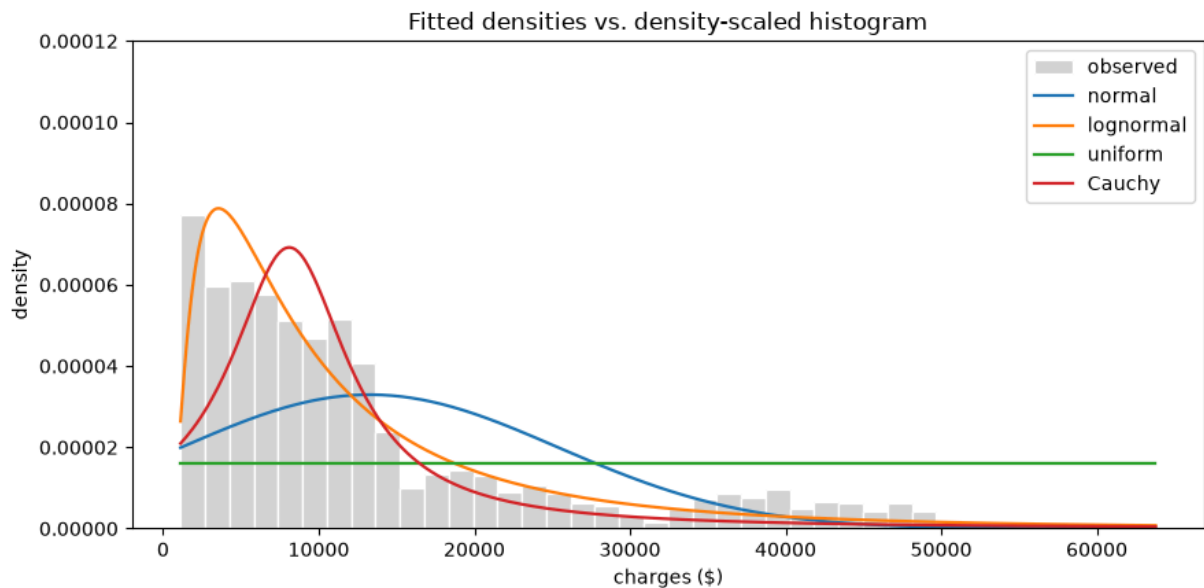
	parameters	log-likelihood
distribution		
normal	1.327e+04, 1.211e+04	-14477.6316
lognormal	0.9828, 378.6, 8349	-13957.3692
uniform	1122, 6.265e+04	-14778.6059
Cauchy	8102, 4600	-14356.3797

In [33]: `grid = np.linspace(charges.min(), charges.max(), 500)`

```

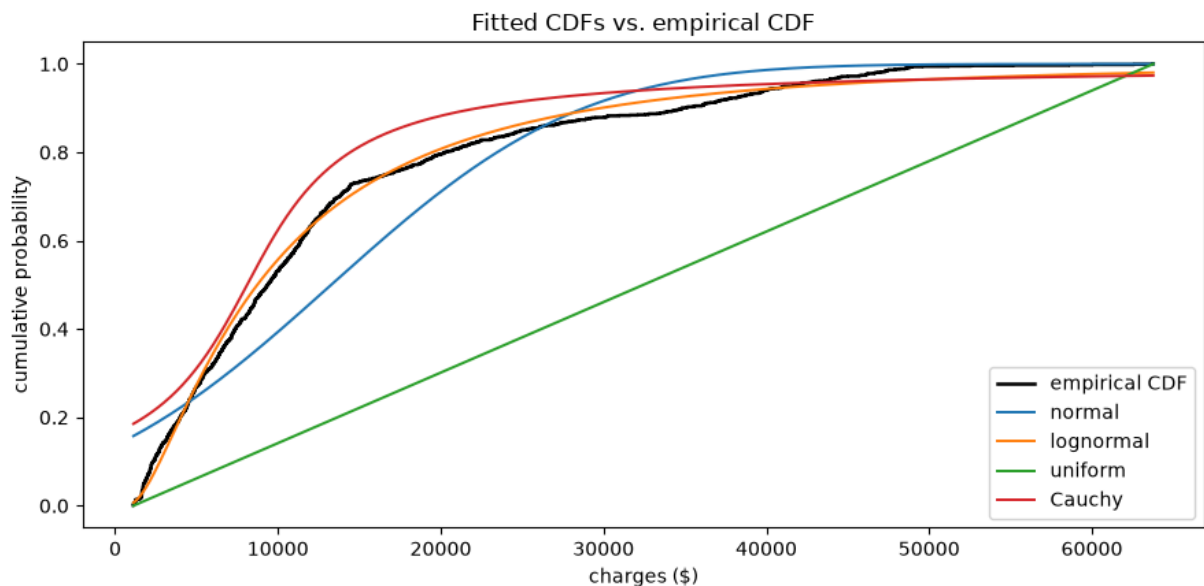
plt.figure(figsize=(9, 4.5))
plt.hist(charges, bins=40, density=True, color="lightgrey", edgecolor="white",
label="observed")
for name, (dist, par) in fits.items():
    plt.plot(grid, dist.pdf(grid, *par), lw=1.6, label=name)
plt.ylim(0, 1.2e-4)
plt.xlabel("charges ($)"); plt.ylabel("density")
plt.title("Fitted densities vs. density-scaled histogram")
plt.legend(); plt.tight_layout(); plt.show()

```



```
In [34]: x_sorted = np.sort(charges)
ecdf = np.arange(1, len(x_sorted) + 1) / len(x_sorted)

plt.figure(figsize=(9, 4.5))
plt.step(x_sorted, ecdf, where="post", color="black", lw=1.8, label="empirical
CDF")
for name, (dist, par) in fits.items():
    plt.plot(grid, dist.cdf(grid, *par), lw=1.4, label=name)
plt.xlabel("charges ($)"); plt.ylabel("cumulative probability")
plt.title("Fitted CDFs vs. empirical CDF")
plt.legend(); plt.tight_layout(); plt.show()
```



```
In [35]: d_ln, p_ln = fits["lognormal"]
d_ca, p_ca = fits["Cauchy"]
d_no, p_no = fits["normal"]

print(f"lognormal P(8k < X < 14k) = {d_ln.cdf(14000, *p_ln) - d_ln.cdf(8000,
*p_ln):.4f}")
```

```
f" (normal {p_norm:.4f}, observed {inwin.mean():.4f})"
print(f"\nnormal P(X > 50k)      = {1 - d_no.cdf(50000, *p_no):.5f} (max
observed = ${charges.max():.0f})")
print(f"Cauchy P(X < 0)          = {d_ca.cdf(0, *p_ca):.4f}")
print(f"Cauchy CDF at max charge = {d_ca.cdf(charges.max(), *p_ca):.4f} ")
```

lognormal P(8k < X < 14k) = 0.2277 (normal 0.1923, observed 0.2825)

normal P(X > 50k) = 0.00121 (max observed = \$63,770)
 Cauchy P(X < 0) = 0.1644
 Cauchy CDF at max charge = 0.9738

The lognormal fits best — Visually evident from plots, and from the log-likelihood ~400 points higher than the other distributions.

- **Centre.** The histogram shows that the data are heavily right skewed, therefore the histogram falls steadily after about \$12,000. The lognormal best reproduces that shape. The normal peaks at \$13.3k, well right of the data's mode, and is far too flat across the whole middle. The uniform is flat by construction. The Cauchy captures better the peak than the normal distribution, and its longer tails fit the distribution better.
- **Tails.** The normal puts only 0.1% of its mass above \$50k although charges reach \$63,770, and 13.7% below zero. The Cauchy fails oppositely — its tails are so heavy that its CDF reaches only 0.974 at the largest observed charge, and it puts 16.4% below zero. The lognormal is supported on $(0, \infty)$ and tracks the empirical CDF closely throughout, running only slightly light in the extreme upper tail where the smoker cluster sits.

Refining (d). The gap in (d) is a *model* failure, not sampling noise. Under the fitted lognormal, $P(\$8k < X < \$14k) = \mathbf{0.2277}$ versus the normal's 0.1923 and the observed 0.2825, hence the lognormal is much closer. It still falls short, and that residual gap is informative: even the lognormal is one unimodal family describing a mixture of smokers and non-smokers, so no off-the-shelf distribution will be exactly right. The fix is to condition on the attributes generating the two clusters. This is what the regression in (f) does.

(f) Regression on all attributes

In [36]:

```
full = smf.ols("charges ~ age + C(sex) + bmi + children + C(smoker) + C(region)",
data=ins).fit()
print(full.summary())
```

```
=====
                        OLS Regression Results
=====
Dep. Variable:          charges    R-squared:                0.751
Model:                  OLS      Adj. R-squared:             0.749
Method:                 Least Squares    F-statistic:          500.8
Date:                  Wed, 23 Sep 2026    Prob (F-statistic):    0.00
Time:                  10:47:29    Log-Likelihood:       -13548.
No. Observations:      1338    AIC:                  2.711e+04
Df Residuals:          1329    BIC:                  2.716e+04
Df Model:              8
Covariance Type:       nonrobust
=====
```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	-1.194e+04	987.819	-12.086	0.000	-1.39e+04	-1e+04
C(sex)[T.male]	-131.3144	332.945	-0.394	0.693	-784.470	521.842
C(smoker)[T.yes]	2.385e+04	413.153	57.723	0.000	2.3e+04	2.47e+04
C(region)[T.northwest]	-352.9639	476.276	-0.741	0.459	-1287.298	581.370
C(region)[T.southeast]	-1035.0220	478.692	-2.162	0.031	-1974.097	-95.947
C(region)[T.southwest]	-960.0510	477.933	-2.009	0.045	-1897.636	-22.466

```

age                256.8564    11.899    21.587    0.000    233.514    280.199
bmi                339.1935    28.599    11.860    0.000    283.088    395.298
children           475.5005    137.804    3.451    0.001    205.163    745.838
=====
Omnibus:           300.366    Durbin-Watson:      2.088
Prob(Omnibus):     0.000    Jarque-Bera (JB):    718.887
Skew:              1.211    Prob(JB):            7.86e-157
Kurtosis:          5.651    Cond. No.            311.
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

(g) Explaining the pricing strategy to a regulator

Categorical predictors are entered with `C(...)`. A variable with k levels uses $k-1$ dummies and the omitted variable is the **reference category**, and each coefficient is a change in charges *relative to that reference*. Reference categories are **female, non-smoker, northeast**, so each coefficient is a dollar difference relative to a female non-smoker in the northeast with the same age, BMI and number of children. The model explains $R^2 = 0.751$ of the variation in charges.

Variable	Coefficient	p-value	Reading
smoker = yes	+\$23,848.53	0.000	dominant term
age	+\$256.86 / year	0.000	steady actuarial gradient
bmi	+\$339.19 / unit	0.000	moderate
children	+\$475.50 / child	0.001	small but significant
sex = male	-\$131.31	0.693	not significant
region = northwest	-\$352.96	0.459	not significant
region = southeast	-\$1,035.02	0.031	significant
region = southwest	-\$960.05	0.045	significant

- **Smoking dominates.** A smoker is associated with a higher charge of nearly \$24,000 more than an otherwise identical non-smoker — *more than the average total charge of \$13,270* ($t = 57.7$).
- **Age** rises steadily and linearly: a 60-year-old is associated with a payment ~\$10,270 more expensive than a 20-year-old, all else equal.
- **BMI:** moving from BMI 25 to BMI 35 is associated with an increase of ~\$3,392.
- **Children:** three dependents is associated with an increase of ~\$1,427.
- **Sex is not being priced.** The estimate is tiny against a \$12,110 standard deviation, and its 95% confidence interval $[-784, +522]$ contains zero, indeed the variable is not significant at the 5% level.
- **Region:** a modest ~\$1,000 is associated with the two southern regions relative to the northeast; the northwest is indistinguishable from it. Both southern p-values sit near the
-

0.05 boundary, so we would not build much on them.

Caveat for the regulator. These are associations in the company's own pricing data, not causal effects, and the p-values assume residuals that are normal with constant variance. Part (i) suggest that this assumption might fail.

```
In [37]: print("95% confidence intervals:")
print(full.conf_int().round(1).rename(columns={0: "2.5%", 1: "97.5%"}))
```

```
95% confidence intervals:
                2.5%    97.5%
Intercept      -13876.4 -10000.7
C(sex)[T.male]   -784.5    521.8
C(smoker)[T.yes] 23038.0 24659.0
C(region)[T.northwest] -1287.3 581.4
C(region)[T.southeast] -1974.1 -95.9
C(region)[T.southwest] -1897.6 -22.5
age             233.5    280.2
bmi             283.1    395.3
children       205.2    745.8
```

(h) Refitting without age

```
In [38]: no_age = smf.ols("charges ~ C(sex) + bmi + children + C(smoker) + C(region)",
data=ins).fit()
print(no_age.summary())
```

```
=====
                        OLS Regression Results
=====
Dep. Variable:          charges    R-squared:                0.664
Model:                  OLS      Adj. R-squared:             0.662
Method:                 Least Squares    F-statistic:         374.8
Date:                  Wed, 23 Sep 2026    Prob (F-statistic):    2.97e-309
Time:                  10:47:29    Log-Likelihood:       -13749.
No. Observations:      1338    AIC:                  2.751e+04
Df Residuals:          1330    BIC:                  2.756e+04
Df Model:               7
Covariance Type:       nonrobust
=====
               coef      std err          t      P>|t|      [0.025      0.975]
-----
Intercept      -3952.9723    1064.068      -3.715    0.000     -6040.407     -1865.538
C(sex)[T.male]   -310.8509     386.671     -0.804    0.422     -1069.403      447.701
C(smoker)[T.yes]  2.366e+04    479.863    49.303    0.000      2.27e+04      2.46e+04
C(region)[T.northwest] -392.3686    553.299     -0.709    0.478     -1477.802      693.065
C(region)[T.southeast] -1410.2693    555.743     -2.538    0.011     -2500.498     -320.040
C(region)[T.southwest] -1031.8915    555.215     -1.859    0.063     -2121.083      57.300
bmi             411.2700     32.998     12.464    0.000      346.537      476.003
children       597.4632     159.956      3.735    0.000      283.669      911.257
=====
Omnibus:            162.572    Durbin-Watson:           2.043
Prob(Omnibus):      0.000    Jarque-Bera (JB):        258.818
Skew:               0.833    Prob(JB):                6.28e-57
Kurtosis:           4.366    Cond. No.                178.
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
In [39]: print(f"R2 with age      : {full.rsquared:.4f}")
print(f"R2 without age : {no_age.rsquared:.4f}")
print(f"change          : {no_age.rsquared - full.rsquared:+.4f}"
      f"   ({(no_age.rsquared - full.rsquared) / full.rsquared:+.1%} relative)")

# how well the other predictors could stand in for age
print("\ncorrelation of age with the other numeric predictors:")
print(ins[["age", "bmi", "children"]].corr()["age"].round(3).to_string())
```

```

ins["resid_no_age"] = no_age.resid
bands = pd.cut(ins["age"], [17, 29, 39, 49, 59, 65])
print("\nmean residual of the no-age model by age band (positive =
undercharged):")
print(ins.groupby(bands, observed=True)["resid_no_age"].agg(["mean",
"count"]).round(1))

```

```

R2 with age      : 0.7509
R2 without age   : 0.6636
change           : -0.0873 (-11.6% relative)

```

correlation of age with the other numeric predictors:

```

age      1.000
bmi      0.109
children 0.042

```

mean residual of the no-age model by age band (positive = undercharged):

```

      mean  count
age
(17, 29] -3602.6   417
(29, 39] -2176.6   257
(39, 49]   511.0   279
(49, 59]  4153.7   271
(59, 65] 6959.8   114

```

Effect on R^2 . It falls $0.7509 \rightarrow 0.6636$, a loss of **0.0873** (11.6% relative). This direction is guaranteed: in-sample R^2 can only increase when a variable is added, so dropping age must lower it. The size of the drop is the point: since the remaining variables cannot absorb age's information, because age is essentially uncorrelated with BMI (0.109) and children (0.042), nothing left in the model proxies for it to mitigate the drop.

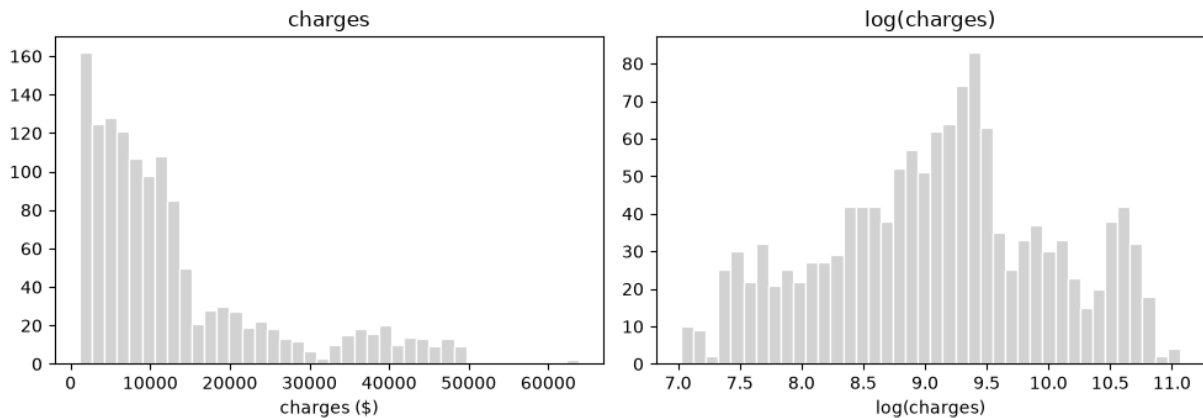
Who is over- and undercharged. Without age, every policyholder with the same remaining attributes gets the same predicted charge — effectively the portfolio average across ages. Since true charges rise steadily with age, the young are pushed *up* toward that average and the old pulled *down*. A positive residual means actual exceeds predicted, i.e. undercharged:

Age band	Mean residual	n	Verdict
18–29	–\$3,603	417	overcharged
30–39	–\$2,177	257	overcharged
40–49	+\$511	279	about right
50–59	+\$4,154	271	undercharged
60–65	+\$6,960	114	undercharged

(i) Should the response be transformed?

```
In [40]: ins["log_charges"] = np.log(ins["charges"])

fig, ax = plt.subplots(1, 2, figsize=(10, 3.6))
ax[0].hist(ins["charges"], bins=40, color="lightgrey", edgecolor="white")
ax[0].set_title("charges"); ax[0].set_xlabel("charges ($)")
ax[1].hist(ins["log_charges"], bins=40, color="lightgrey", edgecolor="white")
ax[1].set_title("log(charges)"); ax[1].set_xlabel("log(charges)")
plt.tight_layout(); plt.show()
```



```
In [41]: fitted = full.fittedvalues
print(f"smallest predicted charge from part (f) : ${fitted.min():.2f}")
print(f"policyholders predicted a negative charge: {(fitted < 0).sum()} of {len(fitted)}")
print(f"smallest charge actually observed : ${ins['charges'].min():.2f}")
```

```
smallest predicted charge from part (f) : $-2,032.91
policyholders predicted a negative charge: 20 of 1338
smallest charge actually observed : $1,121.87
```

```
In [42]: log_model = smf.ols("log_charges ~ age + C(sex) + bmi + children + C(smoker) + C(region)",
                             data=ins).fit()

print(f"\nlog model summary:")
print(log_model.summary())

print(pd.DataFrame({
    "part (f): $ per unit": full.params,
    "log model: coef": log_model.params,
    "multiplier exp(coef)": np.exp(log_model.params),
    "p-value (log model)": log_model.pvalues,
}).drop("Intercept").round(4).to_string())

back = np.exp(log_model.fittedvalues)
print(f"\nback-transformed predictions: ${back.min():.0f} to ${back.max():.0f}, none negative")
```

log model summary:

OLS Regression Results

```

=====
Dep. Variable:      log_charges  R-squared:          0.768
Model:              OLS         Adj. R-squared:     0.767
Method:             Least Squares  F-statistic:       549.8
Date:               Wed, 23 Sep 2026  Prob (F-statistic):   0.00
Time:               10:47:29      Log-Likelihood:    -808.52
No. Observations:   1338         AIC:                1635.
Df Residuals:       1329         BIC:                1682.
Df Model:           8
Covariance Type:    nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	7.0306	0.072	97.112	0.000	6.889	7.173
C(sex)[T.male]	-0.0754	0.024	-3.091	0.002	-0.123	-0.028
C(smoker)[T.yes]	1.5543	0.030	51.333	0.000	1.495	1.614
C(region)[T.northwest]	-0.0638	0.035	-1.827	0.068	-0.132	0.005
C(region)[T.southeast]	-0.1572	0.035	-4.481	0.000	-0.226	-0.088
C(region)[T.southwest]	-0.1290	0.035	-3.681	0.000	-0.198	-0.060
age	0.0346	0.001	39.655	0.000	0.033	0.036
bmi	0.0134	0.002	6.381	0.000	0.009	0.017
children	0.1019	0.010	10.085	0.000	0.082	0.122

```

=====
Omnibus:            463.882  Durbin-Watson:          2.046
Prob(Omnibus):      0.000   Jarque-Bera (JB):        1673.760
Skew:               1.679   Prob(JB):                 0.00
Kurtosis:           7.330   Cond. No.:                 311.
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

	part (f): \$ per unit	log model: coef	multiplier exp(coef)	p-value (log model)
C(sex)[T.male]	-131.3144	-0.0754	0.9274	0.0020
C(smoker)[T.yes]	23848.5345	1.5543	4.7319	0.0000
C(region)[T.northwest]	-352.9639	-0.0638	0.9382	0.0679
C(region)[T.southeast]	-1035.0220	-0.1572	0.8545	0.0000
C(region)[T.southwest]	-960.0510	-0.1290	0.8790	0.0002
age	256.8564	0.0346	1.0352	0.0000
bmi	339.1935	0.0134	1.0135	0.0000
children	475.5005	0.1019	1.1072	0.0000

back-transformed predictions: \$2,278 to \$81,492, none negative

Yes: model log(charges) rather than charges

Part (e) found charges are well described by a lognormal distribution: strongly right-skewed and supported only on positive values. Both consequences for part (f) are visible above.

A possible solution to investigate is engineer the new independent variable log(charges).

This does not guarantee to fix all the problems of the prior attempts, however it would be worth investigating because of the following:

log(charges) is close to symmetric, which is what "lognormal" does: the logarithm is the scale on which the variable is naturally normal. The raw-scale model is not held above zero. It predicts a negative charge for 20 policyholders, down to -\$2,033, when the cheapest real policyholder pays \$1,121.87. Back-transformed predictions from the log model run \$2,278 to \$81,492 and cannot be negative by construction.

This keeps the model a linear regression: "linear" requires only that the model be linear in its coefficients, and $\log(y) = b_0 + b_1x_1 + \dots$ satisfies that.

What we could try: add a column `ins["log_charges"] = np.log(ins["charges"])`, refit the same specification as `smf.ols("log_charges ~ age + C(sex) + bmi + children + C(smoker) + C(region)", data=ins)`, read each coefficient as a roughly percentage change in charges rather than a dollar amount, and apply `np.exp()` to the fitted values to bring predictions back to dollars.

What changes in interpretation. Effects become multiplicative rather than additive. Age's + \$256.86 per year becomes +0.0346, about +3.5% per year; children's + \$475.50 becomes about +10.7% per child. For a coefficient as large as smoking's the multiplier is clearer:

1.5543 means a smoker's charges are multiplied by about 4.7. That is a more useful statement for pricing than a flat +\$23,849 applied regardless of the policyholder's baseline risk.

One consequence the regulator in (g) would need to know. The transform is not cosmetic. On the log scale sex becomes significant ($p = 0.002$, males about 7% cheaper), where on the dollar scale it was not ($p = 0.693$). A flat -\$131 is negligible against a \$12,110 spread, but a consistent -7% is detectable once the model works proportionally. So the part (g) conclusion that the company is not pricing on sex is specific to the dollar-scale model and would have to be revisited.

Therefore, the transform fixes the skew and the positivity of the charges, not the two-group structure part (e) identified: averaging \$8,434 against smokers averaging \$32,050. That is handled by smoker already being a predictor, not by the transformation.

Problem 3 — `mtcars`: interactions and mean-centering (25 points)

```
In [31]: mtcars = sm.datasets.get_rdataset("mtcars", "datasets").data
print(mtcars.shape)
print(f"mean(hp) = {mtcars.hp.mean():.4f}    mean(wt) = {mtcars.wt.mean():.4f}    mean(mpg) = {mtcars.mpg.mean():.4f}")
mtcars.head()
```

```
(32, 11)
mean(hp) = 146.6875    mean(wt) = 3.2173    mean(mpg) = 20.0906
```

```
Out[31]:
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
rownames											
Mazda RX4	21.0000	6	160.0000	110	3.9000	2.6200	16.4600	0	1	4	4
Mazda RX4 Wag	21.0000	6	160.0000	110	3.9000	2.8750	17.0200	0	1	4	4
Datsun 710	22.8000	4	108.0000	93	3.8500	2.3200	18.6100	1	1	4	1
Hornet 4 Drive	21.4000	6	258.0000	110	3.0800	3.2150	19.4400	1	0	3	1
Hornet Sportabout	18.7000	8	360.0000	175	3.1500	3.4400	17.0200	0	0	3	2

(a) Model 1 — `mpg ~ hp + wt + hp:wt`

```
In [32]: model1 = smf.ols("mpg ~ hp * wt", data=mtcars).fit()
print(model1.summary())
```

```
OLS Regression Results
```

	coef	std err	t	P> t	[0.025	0.975]
Dep. Variable:	mpg					
Model:	OLS					
Method:	Least Squares					
Date:	Mon, 21 Sep 2026					
Time:	20:15:56					
No. Observations:	32					
Df Residuals:	28					
Df Model:	3					
Covariance Type:	nonrobust					
R-squared:	0.885					
Adj. R-squared:	0.872					
F-statistic:	71.66					
Prob (F-statistic):	2.98e-13					
Log-Likelihood:	-67.805					
AIC:	143.6					
BIC:	149.5					
Intercept	49.8084	3.605	13.816	0.000	42.424	57.193
hp	-0.1201	0.025	-4.863	0.000	-0.171	-0.070
wt	-8.2166	1.270	-6.471	0.000	-10.818	-5.616
hp:wt	0.0278	0.007	3.753	0.001	0.013	0.043
Omnibus:	2.221					
Prob(Omnibus):	0.329					
Skew:	0.407					
Kurtosis:	2.200					
Durbin-Watson:	2.128					
Jarque-Bera (JB):	1.736					
Prob(JB):	0.420					
Cond. No.	6.35e+03					

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The condition number is large, 6.35e+03. This might indicate that there are strong multicollinearity or other numerical problems.

(b) Interpreting Model 1's coefficients

Model 1 is

$$\widehat{\text{mpg}} = 49.81 - 0.1201 \text{ hp} - 8.2166 \text{ wt} + 0.0278 (\text{hp} \times \text{wt})$$

With the interaction in the model, the "main effect" is the effect when the other predictor equals zero, not the average effect.

- $\text{hp} = -0.1201$. For a car of weight zero, each extra horsepower is associated with a 0.12 mpg decrease. Since no car weighs 0 lbs (the lightest in the data is 1,513 lbs), this is not directly interpretable on its own. It is, however, the starting point of the hp slope, which the interaction then adjusts upward with weight.
- $\text{wt} = -8.2166$. For a car with zero horsepower, each additional 1,000 lbs is associated with an 8.22 mpg decrease. As above this is not directly interpretable on its own.
- $\text{hp:wt} = +0.0278$. This means the effect of horsepower on fuel economy depends on the car's weight: for each extra 1,000 lbs of weight, the marginal penalty of one additional horsepower becomes **0.0278 mpg less severe**. Symmetrically, for each extra horsepower, the mpg penalty per 1,000 lbs of weight is reduced by 0.0278. It is significant ($p = 0.001$).

Overall, the marginal effect of horsepower is

$\partial \text{mpg} / \partial \text{hp} = -0.1201 + 0.0278 \text{ wt}$: -0.078 mpg/hp for a 1,500 lb car but -0.037 mpg/hp for a heavy 3,000 lb one. What I take from that is the logic that heavy cars are already fuel-inefficient, so adding power costs them proportionally less, while in a light, efficient car extra horsepower hurts economy much more.

(c) Model 2 — mean-centered predictors

```
In [34]: mtcars["hpc"] = mtcars["hp"] - mtcars["hp"].mean()
mtcars["wtc"] = mtcars["wt"] - mtcars["wt"].mean()

model2 = smf.ols("mpg ~ hpc * wtc", data=mtcars).fit()
print(model2.summary())
```

```
OLS Regression Results
=====
Dep. Variable:          mpg      R-squared:                0.885
Model:                  OLS      Adj. R-squared:            0.872
Method:                 Least Squares      F-statistic:          71.66
Date:                   Mon, 21 Sep 2026    Prob (F-statistic):    2.98e-13
Time:                   20:15:56           Log-Likelihood:       -67.805
No. Observations:       32              AIC:                 143.6
Df Residuals:           28              BIC:                 149.5
Df Model:                3
Covariance Type:        nonrobust
=====
               coef      std err          t      P>|t|      [0.025      0.975]
-----
Intercept    18.8984      0.496     38.124     0.000     17.883     19.914
hpc          -0.0305      0.008    -4.066     0.000     -0.046     -0.015
wtc          -4.1316      0.530    -7.802     0.000     -5.216     -3.047
hpc:wtc       0.0278      0.007     3.753     0.001      0.013      0.043
=====
Omnibus:            2.221   Durbin-Watson:           2.128
Prob(Omnibus):      0.329   Jarque-Bera (JB):         1.736
Skew:               0.407   Prob(JB):                 0.420
Kurtosis:           2.200   Cond. No.                  97.3
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Comparing Model 2 with Model 1

Model 2 is

$$\widehat{\text{mpg}} = 18.90 - 0.0305 \text{ hp}_c - 4.1316 \text{ wt}_c + 0.0278 (\text{hp}_c \times \text{wt}_c)$$

Coefficient	Model 1 (raw)	Model 2 (centered)
Intercept	49.8084	18.8984
hp / hp_c	-0.1201	-0.0305
wt / wt_c	-8.2166	-4.1316
interaction	0.0278	0.0278
R ²	0.88476	0.88476

The two models are basically the same model since centering is a linear reparameterisation: R^2 , the residual sum of squares, the fitted values and all predictions are identical, the parameterisation of the main effects changes.

- The interaction coefficient is unchanged (0.0278), with the identical standard error, t-statistic and p-value = 0.001 The interaction measures a curvature in the response surface, which does not depend on where the origin sits.
- The main effects change even though they are still "the effect when the other predictor is zero", but zero now means *at the sample mean*. So -0.0305 is the effect of one extra horsepower for a car of average weight (3,217 lbs), and -4.1316 is the effect of 1,000 extra lbs for a car of average power (146.7 hp). Algebraically we can see that: $-0.1201 + 0.0278 \times 3.2173 = -0.0305$ ✓ and $-8.2166 + 0.0278 \times 146.69 = -4.1316$ ✓.
- The intercept becomes meaningful since 18.90 mpg is now the predicted fuel economy of a car with average horsepower and average weight, close to the sample mean mpg of 20.09.
- Collinearity with the product term is reduced, as we can see in Model 1 the raw `hp` and `wt` are strongly collinear with `hp:wt` (variance inflation factors of 19.2, 10.3 and 43.1); after centering the same three VIFs fall to 1.77, 1.80 and 1.02. This does not automatically make both main effects more "precise", we see that `wt` gains ($t = -6.47 \rightarrow -7.80$) while `hp` loses a little ($t = -4.86 \rightarrow -4.07$). In general, indeed, centering does not add or change the information but re-expresses the same fit around a point inside the data, which changes what each main effect is estimating and hence its precision.

(d) Model 3 — interaction only, no main effects

```
In [36]: model3 = smf.ols("mpg ~ hpc:wtc", data=mtcars).fit()
print(model3.summary())
```

```

=====
                        OLS Regression Results
=====
Dep. Variable:          mpg      R-squared:                0.019
Model:                  OLS      Adj. R-squared:           -0.014
Method:                 Least Squares      F-statistic:        0.5771
Date:                  Mon, 21 Sep 2026      Prob (F-statistic):    0.453
Time:                  20:15:56      Log-Likelihood:       -102.07
No. Observations:      32      AIC:                  208.1
Df Residuals:          30      BIC:                  211.1
Df Model:               1
Covariance Type:       nonrobust
=====
                        coef      std err          t      P>|t|      [0.025      0.975]
-----
Intercept      19.4169      1.392      13.950      0.000      16.574      22.260
hpc:wtc         0.0157      0.021       0.760      0.453      -0.027      0.058
=====
Omnibus:                 0.494      Durbin-Watson:          1.068
Prob(Omnibus):           0.781      Jarque-Bera (JB):        0.476
Skew:                    0.263      Prob(JB):                0.788
Kurtosis:                2.717      Cond. No.                87.2
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Comparing Model 3 with Models 1 and 2

Model 3 is $\widehat{\text{mpg}} = 19.4169 + 0.0157(\text{hp}_c \times \text{wt}_c)$.

	Model 1	Model 2	Model 3
Interaction coefficient	0.0278	0.0278	0.0157
Interaction <i>p</i> -value	0.0008	0.0008	0.4534
R ²	0.8848	0.8848	0.0189
Adjusted R ²	0.8724	0.8724	-0.0138

The interaction coefficient changes and becomes not significant, in Models 1 and 2 it was 0.0278 with $p = 0.001$; in Model 3 it is 0.0157 with $p = 0.45$. Centering did not change the interaction. Removing the main effects did change it, though. What happened is that we introduced some bias since the centered product is only weakly correlated with the main effects it now replaces ($r = 0.05$ with `hp_c`, 0.13 with `wt_c`), so the omitted main effects pull the estimate only part-way down, $0.0278 \rightarrow 0.0157$. Moreover, with the main effects gone, almost none of the variation in mpg is explained, so the residual standard error jumps from 2.15 to 6.07 mpg and the interaction's standard error nearly triples, $0.0074 \rightarrow 0.0207$. The *t*-statistic falls from 3.75 to 0.76 because the denominator exploded. In general we have an interaction but no baseline, so it's hard to estimate the interaction itself. In fact, we don't have explanatory power anymore since R^2 falls from 0.885 to 0.019, and the adjusted R^2 is negative (-0.014), meaning this model is worse than simply predicting the sample mean for every car. The intercept, 19.42, is close to the mean mpg of 20.09, which is exactly what one expects from a model with no real predictive content.

Cambridge Computers: Predicting Laptop Price

Problem 4 — Logistic Regression

Management now wants to classify a laptop as **high-priced** (`Price >= 500`) or not.

Problem 4 setup: create the binary outcome **high**

`high = 1` if `Price >= 500` Euros, `high = 0` otherwise, in **both** train and test.

```
In [14]: train["high"] = (train["Price"] >= 500).astype(int)
test["high"] = (test["Price"] >= 500).astype(int)

print("Train high counts:\n", train["high"].value_counts().sort_index().to_string())
print("\nTest high counts:\n", test["high"].value_counts().sort_index().to_string())
print(f"\nProportion high -- train: {train['high'].mean():.4f} | test: {test['high'].mean():.4f}")
```

Train high counts:

	high
0	115
1	550

Test high counts:

	high
0	53
1	227

Proportion high -- train: 0.8271 | test: 0.8107

Problem 4(a)

Logistic regression predicting `high` from all independent variables in Table 1. **No variable selection is performed.**

```
In [15]: log_formula = "high ~ C(Company) + C(TypeName) + C(GPU) + Screen + Memory + Weight + Rating"
log_model = smf.logit(log_formula, data=train).fit()
print(log_model.summary())
```

Warning: Maximum number of iterations has been exceeded.
 Current function value: 0.251696
 Iterations: 35

Logit Regression Results

Dep. Variable:	high	No. Observations:	665
Model:	Logit	Df Residuals:	653
Method:	MLE	Df Model:	11
Date:	Tue, 22 Sep 2026	Pseudo R-squ.:	0.4534
Time:	17:09:49	Log-Likelihood:	-167.38
converged:	False	LL-Null:	-306.24
Covariance Type:	nonrobust	LLR p-value:	4.281e-53

	coef	std err	z	P> z	[0.025	0.975]
Intercept	29.4861	5959.094	0.005	0.996	-1.17e+04	1.17e+04
C(Company) [T.Dell]	1.2456	0.501	2.486	0.013	0.263	2.228
C(Company) [T.HP]	1.4910	0.465	3.206	0.001	0.579	2.403
C(Company) [T.Lenovo]	-0.0525	0.445	-0.118	0.906	-0.925	0.820
C(TypeName) [T.Notebook]	-17.6031	5959.093	-0.003	0.998	-1.17e+04	1.17e+04
C(TypeName) [T.Ultrabook]	-16.1946	5959.093	-0.003	0.998	-1.17e+04	1.17e+04
C(GPU) [T.Intel]	-0.1053	0.354	-0.298	0.766	-0.799	0.589
C(GPU) [T.Nvidia]	2.0148	0.605	3.329	0.001	0.829	3.201
Screen	-1.1954	0.311	-3.845	0.000	-1.805	-0.586
Memory	0.7421	0.093	8.019	0.000	0.561	0.923
Weight	1.5361	0.903	1.701	0.089	-0.234	3.306
Rating	-0.0904	0.049	-1.838	0.066	-0.187	0.006

Possibly complete quasi-separation: A fraction 0.20 of observations can be perfectly predicted. This might indicate that there is complete quasi-separation. In this case some parameters will not be identified.

/Users/chrislowzx/miniconda3/lib/python3.13/site-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals
 warnings.warn("Maximum Likelihood optimization failed to "

Answer (a). The fitted logistic regression model using all independent variables is shown above. The model produces a convergence warning due to all Gaming laptops in the training data are high-priced. So some TypeName

coefficient estimates should be interpreted with caution.

Problem 4(b)

Which variables are significant in predicting the probability of a high price?

```
In [17]: sig = pd.DataFrame({"coef": log_model.params, "p_value": log_model.pvalues})
sig["signif_5pct"] = np.where(sig["p_value"] < 0.05, "YES", "no")
print(sig.round(4).to_string())

print("\nSignificant at 5%:", [v for v in sig.index[sig["p_value"] < 0.05] if v != "Intercept"])
```

	coef	p_value	signif_5pct
Intercept	29.4861	0.9961	no
C(Company) [T.Dell]	1.2456	0.0129	YES
C(Company) [T.HP]	1.4910	0.0013	YES
C(Company) [T.Lenovo]	-0.0525	0.9061	no
C(TypeName) [T.Notebook]	-17.6031	0.9976	no
C(TypeName) [T.Ultrabook]	-16.1946	0.9978	no
C(GPU) [T.Intel]	-0.1053	0.7661	no
C(GPU) [T.Nvidia]	2.0148	0.0009	YES
Screen	-1.1954	0.0001	YES
Memory	0.7421	0.0000	YES
Weight	1.5361	0.0889	no
Rating	-0.0904	0.0660	no

Significant at 5%: ['C(Company) [T.Dell]', 'C(Company) [T.HP]', 'C(GPU) [T.Nvidia]', 'Screen', 'Memory']

Answer (b). At the 5% level the significant predictors are **Memory** ($p < 0.001$), **Screen** ($p < 0.001$), **GPU = Nvidia** ($p = 0.001$), and **Company = HP** ($p = 0.001$) and **Company = Dell** ($p = 0.013$) relative to the Asus baseline. **Weight** ($p = 0.089$) and **Rating** ($p = 0.066$) are significant only at the 10% level, and **GPU = Intel** and **Company = Lenovo** are not significant.

Note that the **TypeName** p-values (≈ 0.998) are meaningless here because of the separation described in part (a).

Problem 4(c)

Are the significant variables the same as in the Problem 1 linear model?

```
In [18]: compare = pd.DataFrame({
    "linear_coef": lin_model.params,
    "linear_p":    lin_model.pvalues,
    "logit_coef":  log_model.params,
    "logit_p":     log_model.pvalues,
})
compare["linear_sig_5pct"] = np.where(compare["linear_p"] < 0.05, "YES", "no")
compare["logit_sig_5pct"]  = np.where(compare["logit_p"]  < 0.05, "YES", "no")
compare["same_significance"] = np.where(
    compare["linear_sig_5pct"] == compare["logit_sig_5pct"], "same", "DIFFERENT")
print(compare.round(4).to_string())
```

	linear_coef	linear_p	logit_coef	logit_p	linear_sig_5pct	logit_sig_5pct
same_significance						
Intercept	1434.2466	0.0000	29.4861	0.9961	YES	no
DIFFERENT						
C(Company) [T.Dell]	86.7254	0.0438	1.2456	0.0129	YES	YES
same						
C(Company) [T.HP]	170.3954	0.0001	1.4910	0.0013	YES	YES
same						
C(Company) [T.Lenovo]	35.3279	0.4020	-0.0525	0.9061	no	no
same						
C(TypeName) [T.Notebook]	-76.2954	0.1943	-17.6031	0.9976	no	no
same						
C(TypeName) [T.Ultrabook]	416.3939	0.0000	-16.1946	0.9978	YES	no
DIFFERENT						
C(GPU) [T.Intel]	165.4297	0.0000	-0.1053	0.7661	YES	no
DIFFERENT						
C(GPU) [T.Nvidia]	218.1386	0.0000	2.0148	0.0009	YES	YES
same						
Screen	-120.9323	0.0000	-1.1954	0.0001	YES	YES
same						
Memory	87.3725	0.0000	0.7421	0.0000	YES	YES
same						
Weight	271.0986	0.0000	1.5361	0.0889	YES	no
DIFFERENT						
Rating	-11.8206	0.0106	-0.0904	0.0660	YES	no
DIFFERENT						

Answer (c). The significant predictors are not identical between the two models. **Memory, Screen, GPU = Nvidia, Company = Dell, and Company = HP** are significant in both, while **GPU = Intel, Weight, Rating, and TypeName = Ultrabook** are significant only in the linear model.

Problem 4(d)

For each significant variable, does an increase in its value raise or lower the probability of being high-priced?

```
In [19]: signif = compare.loc[(compare["logit_p"] < 0.05) & (compare.index != "Intercept")].copy()
signif["odds_ratio"] = np.exp(signif["logit_coef"])
```

```
signif["direction"] = np.where(signif["logit_coef"] > 0, "INCREASES P(high)", "DECREASES P(high)")
print(signif[["logit_coef", "odds_ratio", "logit_p", "direction"]].round(4).to_string())
```

	logit_coef	odds_ratio	logit_p	direction
C(Company) [T.Dell]	1.2456	3.4750	0.0129	INCREASES P(high)
C(Company) [T.HP]	1.4910	4.4416	0.0013	INCREASES P(high)
C(GPU) [T.Nvidia]	2.0148	7.4991	0.0009	INCREASES P(high)
Screen	-1.1954	0.3026	0.0001	DECREASES P(high)
Memory	0.7421	2.1003	0.0000	INCREASES P(high)

Answer (d).

Among the significant predictors, higher **Memory** increases the probability of being high-priced (coefficient = 0.742, odds ratio = 2.10), which makes sense since additional RAM is generally associated with more expensive laptops.

Nvidia GPUs (coefficient = 2.015, OR = 7.50), **HP** (1.491, OR = 4.44), and **Dell** (1.246, OR = 3.48) also increase the probability relative to their baseline categories, AMD and Asus. This makes sense, as Nvidia GPUs are often connected to higher-performance laptops, while the positive HP and Dell effects suggest that laptops from these manufacturers are more likely to be high-priced than similar Asus laptops in this dataset.

In contrast, a larger **Screen** decreases the probability (coefficient = -1.195, OR = 0.30). This is somewhat counterintuitive since larger screens might be expected to increase price, but the data suggests the opposite after controlling for the other laptop characteristics.

Problem 4(e)

For which variables do the linear and logistic coefficients share a sign, and for which do they differ?

```
In [20]: signs = compare[["linear_coef", "logit_coef", "linear_p", "logit_p"]].copy()
signs["linear_sign"] = np.where(signs["linear_coef"] > 0, "+", "-")
signs["logit_sign"] = np.where(signs["logit_coef"] > 0, "+", "-")
signs["agreement"] = np.where(signs["linear_sign"] == signs["logit_sign"], "SAME", "DIFFERENT")
print(signs.round(4).to_string())
```

ent	linear_coef	logit_coef	linear_p	logit_p	linear_sign	logit_sign	agreem
Intercept	1434.2466	29.4861	0.0000	0.9961	+	+	S
AME							
C(Company) [T.Dell]	86.7254	1.2456	0.0438	0.0129	+	+	S
AME							
C(Company) [T.HP]	170.3954	1.4910	0.0001	0.0013	+	+	S
AME							
C(Company) [T.Lenovo]	35.3279	-0.0525	0.4020	0.9061	+	-	DIFFER
ENT							
C(TypeName) [T.Notebook]	-76.2954	-17.6031	0.1943	0.9976	-	-	S
AME							
C(TypeName) [T.Ultrabook]	416.3939	-16.1946	0.0000	0.9978	+	-	DIFFER
ENT							
C(GPU) [T.Intel]	165.4297	-0.1053	0.0000	0.7661	+	-	DIFFER
ENT							
C(GPU) [T.Nvidia]	218.1386	2.0148	0.0000	0.0009	+	+	S
AME							
Screen	-120.9323	-1.1954	0.0000	0.0001	-	-	S
AME							
Memory	87.3725	0.7421	0.0000	0.0000	+	+	S
AME							
Weight	271.0986	1.5361	0.0000	0.0889	+	+	S
AME							
Rating	-11.8206	-0.0904	0.0106	0.0660	-	-	S
AME							

Answer (e).

The coefficients have the **same sign** for **Company = Dell (+)**, **Company = HP (+)**, **TypeName = Notebook (-)**, **GPU = Nvidia (+)**, **Screen (-)**, **Memory (+)**, **Weight (+)**, and **Rating (-)**.

The signs are **different** for **Company = Lenovo** (+ in linear, - in logistic), **GPU = Intel** (+ in linear, - in logistic), and **TypeName = Ultrabook** (+ in linear, - in logistic). However, again, the **TypeName** logistic coefficient should be interpreted cautiously because of the quasi-complete separation identified in part (a).

Problem 4(f)

Probability that a Lenovo Ultrabook (Intel GPU), Screen = 8, Memory = 8, Weight = 4.2, Rating = 7 is high-priced.

$$P(\text{high} = 1 \mid x) = \frac{1}{1 + e^{-z}}, \quad z = \beta_0 + \beta_{\text{Lenovo}} + \beta_{\text{Ultrabook}} + \beta_{\text{Intel}} + \beta_{\text{Screen}}(8) \\ + \beta_{\text{Memory}}(8) + \beta_{\text{Weight}}(4.2) + \beta_{\text{Rating}}(7)$$

The Dell/HP, Notebook and Nvidia indicators are all 0 for this laptop, so they drop out.

```
In [21]: new_laptop_4 = pd.DataFrame({
    "InventoryID": [4096],
    "Company":     ["Lenovo"],
    "TypeName":    ["Ultrabook"],
    "GPU":         ["Intel"],
    "Screen":      [8.0],
    "Memory":      [8],
    "Weight":      [4.2],
    "Rating":      [7],
    })

b = log_model.params
terms = {
    "Intercept":          b["Intercept"],
    "Company=Lenovo":     b["C(Company) [T.Lenovo]"],
    "TypeName=Ultrabook": b["C(TypeName) [T.Ultrabook]"],
    "GPU=Intel":          b["C(GPU) [T.Intel]"],
    "Screen x 8.0":       b["Screen"] * 8.0,
    "Memory x 8":         b["Memory"] * 8,
    "Weight x 4.2":       b["Weight"] * 4.2,
    "Rating x 7":         b["Rating"] * 7,
}

for k, v in terms.items():
    print(f" {k:<20} = {v:>12.4f}")

z_val = sum(terms.values())
p_manual = 1 / (1 + np.exp(-z_val))
p_smf = log_model.predict(new_laptop_4).iloc[0]
```

```
print(f"\n z (log-odds)           = {z_val:.4f}")
print(f"  P(high=1) manual        = {p_manual:.8f}")
print(f"  P(high=1) via .predict      = {p_smf:.8f}")
print(f"  1 - P(high=1)                = {1 - p_manual:.3e}")
```

```
Intercept           = 29.4861
Company=Lenovo      = -0.0525
TypeName=Ultrabook  = -16.1946
GPU=Intel           = -0.1053
Screen x 8.0        = -9.5635
Memory x 8          = 5.9365
Weight x 4.2        = 6.4516
Rating x 7          = -0.6325
```

```
z (log-odds)        = 15.3257
P(high=1) manual     = 0.99999978
P(high=1) via .predict = 0.99999978
1 - P(high=1)       = 2.209e-07
```

Answer (f).

For this laptop, the fitted log-odds are

$$z = 29.4861 - 0.0525 - 16.1946 - 0.1053 - 1.1954(8) + 0.7421(8) + 1.5361(4.2) - 0.0904(7) \approx 15.33.$$

Therefore,

$$P(\text{high} = 1) = \frac{1}{1 + e^{-15.33}} \approx 0.99999978.$$

Thus, the fitted model predicts an approximately **100% probability** that this laptop is high-priced. This prediction should be interpreted cautiously because the logistic regression had the quasi-complete separation issue identified in part (a).

Problem 4(g)

Apply the model to the test set with a 0.5 probability cutoff and compute accuracy.

```
In [22]: test_probs = log_model.predict(test)
test_class = (test_probs >= 0.5).astype(int)

conf = pd.crosstab(test["high"], test_class, rownames=["Actual"], colnames=["Predicted"])
print("Confusion matrix (test set):")
print(conf.to_string(), "\n")

n_correct = (test_class == test["high"]).sum()
accuracy = n_correct / len(test)
baseline = test["high"].mean() # naive rule: always predict "high"

print(f"Test accuracy (cutoff 0.5) = {accuracy:.4f} ({n_correct} / {len(test)})")
print(f"Baseline accuracy = {baseline:.4f}")
```

Confusion matrix (test set):

Predicted	0	1
Actual		
0	27	26
1	15	212

Test accuracy (cutoff 0.5) = 0.8536 (239 / 280)

Baseline accuracy = 0.8107

Answer (g).

Using a probability cutoff of 0.5, the model correctly classifies **239 out of 280** laptops in the test set. Thus, the test accuracy is

$$\text{Accuracy} = \frac{239}{280} = 0.8536 \approx 85.4\%.$$

This is slightly better than the **81.1% naive baseline** from always predicting a laptop as high-priced.